

BUIDL CTC 2026 FALL - AI TRACK

Attest AI

Verified Agent Guard

Deterministic AI policy. Cryptographically proven source events. An auditable allow/deny registry

Attestcoin Protocol | Sepolia | Creditcoin CC3 Testnet

DoraHacks submission brief - September 2026

Why AI agents need a guardrail

Autonomous agents increasingly trigger payments, credit actions, and infrastructure calls. Most safety layers ask users to trust a model's explanation or a centralized oracle. Attest AI separates explanation from proof: the AI policy is deterministic and versioned, while Attestcoin cryptographically proves that the policy result really happened on Sepolia.

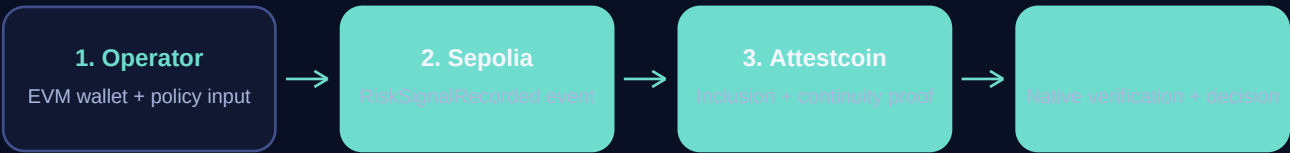
Operator problem

An agent operator must prove to auditors that an action was based on an actual source-chain event, not a fabricated or stale model output. Current logs are mutable, oracle statements are opaque, and LLM explanations cannot be used as on-chain evidence.

Solution

The browser computes Policy v1 from the intended recipient and notional value. The operator signs a RiskSignalRecorded event on Sepolia. After Attestcoin attestation, the app submits the proof to AttestGuard on CC3. The contract verifies the source receipt, emitter, score, evidence hash, policy version, and duplicate intent before recording the final decision.

Architecture and trust model



No backend, no centralized oracle, no custody

Fact	Attest AI treatment
Source event	Verified by Attestcoin inclusion and continuity proof
Risk score	Recomputed by Solidity from the proved event
Evidence hash	Recomputed and compared on CC3
AI explanation	Generated off-chain, but not relied on for trust
Target decision	Public mapping and event on Creditcoin

Deterministic policy v1

Input	Score impact
Base policy	20
Value above 0.05 ETH	+30
Value above 0.5 ETH	+30
Recipient equals signer	+15
Zero-value intent	+10
Final score at or below 60	Allow
Final score above 60	Deny and record for audit

The same constants are implemented in TypeScript and Solidity. Contract tests compare the source and target implementations for low, mid, high, self-directed, and zero-value inputs. The target recomputes the score and evidence hash, so a tampered event cannot pass.

Implementation and verification

Layer	Implementation
Source	SourceRiskSignal.sol on Sepolia
Target	AttestGuard.sol with ASCBase and native verifier
Client	React 19, TypeScript strict, ethers v6
Proofs	Official Attestcoin proof-builder API
Testing	24 automated tests across TypeScript and Solidity
Build	npm run typecheck, npm test, npm run build, forge test

Security checks

- Failed source receipts are rejected.
- Untrusted emitters are rejected.
- Malformed event topics are rejected.
- Tampered scores and evidence hashes are rejected.
- Duplicate intents are rejected.
- Only the contract owner can register the trusted source emitter.

Current testnet evidence

Artifact	Value
Sepolia source contract	0x7CFC5C06aFfBe46c55b9d5313A9ab2A5faa1a2BD
Deployment transaction	0x0bc6fbe5979818906e1587fa32112a999afff19c5f87bb72470cab54e047ca22
High-risk signal	0x6302b379257c1a6aa700b210fd8234b672e4b465805f4d928f3de9555195ce9f
Signal score	80 / 100 - expected deny
Official proof	Saved at docs/evidence/high-risk-proof.json
Low-risk signal	0xd9fe599bbf2f870314d2884250359113998c73ed0b473832db0ab7efca853ef4
Low-risk score	20 / 100 - expected allow
Low-risk proof	Saved at docs/evidence/low-risk-proof.json
CC3 guard	Pending official Discord testnet faucet

Submission status

Source, contracts, client, tests, README, and this brief are complete. Full CC3 deployment and the end-to-end demonstration video are scheduled as soon as the official faucet delivers testnet CTC. All final transaction links will be written into the repository README and the DoraHacks submission form.